

Project 2 长期完善路线

项目定位：面向银行协议问答的、可复现、可评测、可解释、可部署的 RAG / Controlled Agentic RAG 系统

文档版本：v1.1

更新日期：2026-10-01

1. 核心结论

这个项目值得长期改，但不能以“扩大评测集、故意降低 Baseline”为目标。

正确做法是：

- 冻结当前自然评测集，所有新旧版本都在相同数据上比较。
- 单独建立复杂问题开发集和锁定测试集，用于验证 Query Planner。
- 先证明问题存在，再引入对应技术。
- 每个阶段设置进入条件、验收门槛和止损条件。
- LangGraph、MCP、LoRA、vLLM 等技术只在解决真实问题时接入。

最终简历重点不是“使用了多少热门框架”，而是能够说明：

- 为什么使用 BM25 + Dense Hybrid。
- 为什么需要 Reranker。
- 为什么 Hit@K 不足以评价多证据问题。

- Query Planner 为什么可能负优化。
- 如何定位 Planner、Retriever、Reranker、Merge、Generator 的失败。
- 如何防止评测泄漏和测试集过拟合。
- 某项技术为什么进入项目，以及它的收益、代价和止损条件。

2. 当前项目事实基线

当前已验证结果：

模块	当前结果	工程判断
Hybrid Retriever	Hit@5 94%, Complete@5 92%, Complete@20 100%	混合检索主链路成立，应保留
Cross Encoder Reranker	Hit@5 100%, Complete@5 96%, MRR@5 0.824	能拉回后排证据，但可能破坏多证据完整性
多证据样本	7 条	数量不足以稳定判断 Planner 价值
Low Planner Pipeline	Top5 Complete 与 Minimal 相同，MRR 更低	当前没有证明 Planner 收益
平均总延迟	Minimal 1.518 秒，Low 9.347 秒	Planner 当前带来约 6 倍总延迟

因此，当前最重要的问题不是继续增加 Agent 功能，而是回答：

Planner 没有效果，是因为评测集缺少复杂问题、Query 拆分质量不足，还是最终 Fusion/Merge 把已召回证据排坏了？

3. 目标架构

数据面

PDF → 解析与清洗 → 版本化 Chunk → Embedding/BM25/FAISS 索引

↓

评测面

冻结测试集 → Component Eval → End-to-End Eval → Regression Report

↓

查询面

Query

- 输入校验
- Minimal / Planned / Refuse 路由
- Query Plan
- 每个 Query 独立 Hybrid Retrieval
- 每个 Query 独立 Rerank
- Fusion / Coverage Selection
- Evidence Sufficiency
- 最多一次补检索
- Context Budget
- Generation
- Citation Verification
- Output Guard

横切能力

配置、Schema、版本号、Trace、延迟、测试、API、安全

这套路线参考但不照搬以下真实工程：

- Haystack：组件化 Pipeline、分支、循环和 Rank Fusion。
- RAGFlow：可查看的切块、检索测试、引用和空答案。
- Pyserini：可复现的稀疏、稠密、混合检索和标准评价。
- Azure Agentic Retrieval：Minimal、Planning、多查询检索、合并和活动日志。
- LangSmith / Phoenix：离线回归、实验追踪和可观测性。

4. 阶段路线

阶段 0：冻结 Baseline，补齐工程基础

是否必要

必要，且应最先完成。否则后续指标变化无法区分是算法、数据还是环境变化造成的。

工作内容

- 冻结当前 61 条评测数据为 `bank_eval_natural_v1`。
- 保存 corpus、chunker、embedding、reranker、planner、prompt 和配置版本。
- 增加明确的依赖文件与启动说明。
- 将模型名、TopK、阈值等移入统一配置。
- 为 QueryPlan、Chunk、RetrievalResult、PipelineResult 建立 Pydantic Schema。
- 建立 unit、integration、regression 三层测试。
- 保留现有业务类，避免一次性大重构。

验收门槛

- 新环境可根据文档运行完整评测。
- 当前 Baseline 可以重复得到。
- 修改 Planner 不影响 Minimal 模式。
- 每份报告包含数据、模型、Prompt、索引和配置版本。

止损条件

- 不为“代码看起来更规范”进行大范围移动文件。
- 不在此阶段修改检索算法。

面试问题

- 如何保证实验可复现？
- 配置和模型版本如何管理？
- 单元测试、集成测试、回归测试分别测什么？

阶段 1：建立版本化评测体系

是否必要

必要。这是整个长期路线优先级最高的能力。

数据集设计

1. `natural_v1`：当前自然问题，永久冻结，用于检查旧能力退化。
2. `planner_dev`：真实需要跨段、跨协议、条件组合的问题，可用于分析和调参。
3. `planner_test_locked`：不参与调参，只用于阶段验收。
4. `security_regression`：Prompt Injection、越权、超长输入和格式异常。

每条样本至少记录

- Query、task_type、evidence_mode。
- corpus_version。
- 必需证据集合和 Expected Points。
- 样本来源和构造理由。
- 可选的人工 Gold Subqueries。

评价维度

- Retriever: Hit、Recall、Complete、MRR，必要时增加 NDCG/MAP。
- Reranker: 拉回数、破坏数、证据完整性。
- Planner: Schema 合法率、子查询覆盖、Oracle 差距。
- End-to-End: 答案要点、引用、拒答、端到端延迟。
- 系统成本: Query 数、候选数、Reranker Pair 数、各阶段 P50/P95。

验收门槛

- 所有版本在相同冻结数据上比较。
- 同时报告百分比和样本绝对数量。
- Component Eval 与 End-to-End Eval 分离。
- LLM Judge 只作辅助，不替代人工证据和 Expected Points。

止损条件

- 不通过修改旧问题来配合新算法。
- 不把新难题上的低分宣传成旧 Baseline 被改进。

面试问题

- 为什么要区分 Dev 与 Locked Test?
 - 为什么 Hit@K 不足以评价多证据问题?
 - 如何避免评测数据泄漏?
-

阶段 2：版本化数据处理与索引

工作内容

- 使用稳定标识：`document_id + document_version + chunk_ordinal + content_hash`。
- 为每次构建生成 corpus/index manifest。
- 持久化 Embedding、FAISS 和 BM25 索引。
- Chunk 能追溯至 PDF、页码或原段落。
- 生成切块审计报告。
- 对自然段、重叠、标题感知切分做相同数据上的对照。
- 重新切块时根据真实文本迁移标签，禁止沿用旧数字 ID。

暂时不做

- 不迁移向量数据库。当前数据量很小，尚无动态更新、多租户、并发写入和复杂过滤需求。

验收门槛

- 相同 corpus 版本得到相同 Chunk 和索引。
- 应用启动不再重新计算全部 Embedding。
- 每个引用可以返回原文来源。
- 语料变化会触发显式版本变化，而不是静默污染标签。

面试问题

- 为什么 Chunk ID 不能只用递增数字？
- 索引如何版本化与重建？
- 什么时候需要向量数据库？

阶段 3：固化 Retriever 与 Reranker

工作内容

- 保留 Dense、BM25、Hybrid 三组对照。
- TopK、RRF 权重、candidate_k 只在 Dev 集调整。
- 批量计算 Query Embedding。
- 验证 Query/Chunk 截断策略。
- 统计 P50/P95、候选数和 Reranker Pair 数。
- 取消固定 `reranker_score >= 0.9` 的通用业务含义，改为 TopK 或经 Dev 校准的策略。

验收门槛

- Hybrid 稳定优于或至少不弱于单路检索。

- Reranker 拉回和破坏的案例都可解释。
- 每个参数变更都附同数据集对照报告。

面试问题

- BM25 与 Dense 各自解决什么问题？
- RRF 为什么适合融合不同分数量纲？
- 为什么 Cross Encoder 可能损害证据集合完整性？

阶段 4: Oracle 驱动的 Query Planner 与 Fusion 验证

是否必要

这是当前最关键的算法阶段，应先于 LangGraph、MCP 和 LoRA。

四组严格对照

- A. Original Query
- B. 简单规则拆分
- C. LLM Planner
- D. 人工 Oracle Subqueries

四组使用相同 Retriever、Reranker、candidate_k 和最终 K。

必须记录的链路

- 子查询生成
 - 每个子查询 Hybrid Top20
 - 每个子查询 Reranker 结果
 - Fusion 前结果
 - Fusion 后结果

Fusion 对照

- 当前 round-robin。
- RRF。
- Weighted RRF。
- Coverage-aware quota/selector。
- 必要时 distribution-based fusion。

决策树

Oracle 也不优于 Original

→ 停止优化 Planner, 检查语料、检索和任务本身

Oracle 有效, 但 LLM Planner 无效

→ 改 Planner 数据或 Prompt, 未来才考虑 LoRA

子查询已经找到证据, 但 Fusion 后丢失

→ 优化 Fusion/Coverage, 不继续调 Planner Prompt

复杂问题提升, 但简单问题退化

→ Selective Router, 只对复杂问题启用 Planner

验收门槛

- Oracle 对复杂问题存在净拉回。
- Planner 与 Oracle 的差距可以定位。
- 新增收益超过回退案例。
- Planner 收益值得其额外延迟和调用成本。

止损条件

- Oracle 无收益时终止 Planner 主线。
- 不设置 “Planner 必须胜出” 的结论。

面试问题

- 如何证明 Query Decomposition 值得做?
- 为什么要先跑 Oracle?
- Rank Fusion 和 Evidence Selection 有什么区别?

阶段 5: 受控 Agentic RAG

进入条件

阶段 4 已证明 Planner 或多查询检索对复杂问题有价值。

目标 Workflow

Minimal: Original Query 检索

Low: Planner → 多 Query 检索 → Fusion

Medium: Low → Evidence Sufficiency → 最多一次补检索

Refuse: 证据不足或超出语料范围

工作内容

- QueryPlan 使用 Pydantic Structured Output。
- 设置最大子查询数、最大候选数、最大 Reranker Pair 数。
- 最多补检索一次。
- 记录每一步活动和失败原因。
- 银行领域不设置“模型凭内部知识直接回答”的路由。
- 使用 Context Budget 控制重复证据和 Token 数量。

不做

- 不实现无限 ReAct Loop。
- 不让 Agent 自由创建工具或改变系统配置。

验收门槛

- Minimal、Low、Medium 能在同一评测中对比。
- 每次 Retry 都能解释触发原因。
- 超出预算时能降级或拒答。

对应学习计划

- Structured Output / Function Calling。
- Context Engineering。
- Agentic RAG / Planning / Routing。
- Agent Evaluation / Reliability。
- ReAct 只学习思想，不作为无边界生产架构。

手写 Tool Calling 基础闭环

这部分应当加入项目，但它首先是一个框架无关的应用编排能力，不能为了展示 Function Calling，把原本固定的单次检索强行改成 LLM 选工具。

第一版按照下面的闭环实现：

1. 写 Python Function
2. 定义 Tool Schema
3. 把 Tools 传给 LLM
4. 第一次调用 LLM
5. 读取零个、一个或多个 Tool Call
6. Tool Registry 校验名称和参数，然后执行函数
7. 使用 tool_call_id / call_id 将 Tool Result 回填对话
8. 第二次调用 LLM，得到最终答案或下一轮 Tool Call

真实工程中，第 8 步不能假设必然返回最终文本：模型可能不调用工具，也可能一次调用多个工具，或继续请求下一轮工具。因此第一版应设置 `max_tool_rounds=1`，超过预算就停止、降级或报错；后续只有评测证明需要时才增加轮数。

项目中的最小 Tool 集合建议为：

```
search_bank_documents(query, top_k, filters=None)
get_document_metadata(document_id)
```

`search_bank_documents` 只暴露已经稳定的 Retrieval Pipeline，不把 Vector、BM25、Reranker 分别交给 LLM 自由编排。`get_document_metadata` 用于查询来源、版本和协议信息。只有存在两个以上职责不同、确实需要模型选择的工具时，Tool Calling 才比固定 Python 调用链更有意义。

Tool Registry 至少负责：

- 维护工具名称到 Python callable 的显式白名单映射。
- 使用 Pydantic/JSON Schema 校验输入，不直接相信模型生成的 arguments。
- 禁止使用 `eval`、`globals()[name]` 或模型指定的动态 import。
- 为每次调用保存 `tool_call_id`、`tool_name`、arguments、latency、status。
- 处理未知工具、非法参数、函数异常、超时和过大输出。
- 对 Tool Result 做输出 Schema 校验和长度限制，并将其视为不可信数据。
- 第一版顺序执行多个 Tool Call；只有存在真实延迟问题时再评估并行执行。

官方 Function Calling 流程同样是“发送工具 → 接收调用 → 应用侧执行 → 回填带 call_id 的结果 → 再次调用模型”。工具参数由 JSON Schema 描述，应用侧仍负责真正执行函数。MCP 后续只是把相同能力通过标准协议暴露出去，不替代这个应用侧闭环。

Tool Calling 测试至少覆盖：

- 模型不调用任何工具。
- 正确调用一个工具。

- 一次返回多个 Tool Call。
- 工具名不存在。
- arguments 不是合法 JSON 或不满足 Schema。
- 工具超时、抛异常或返回超大内容。
- Tool Result 与正确的 `tool_call_id` 对应。
- 达到最大轮数后能够终止，不发生无限循环。

验收门槛：

- 先使用 Fake LLM/Fake Tool 跑通闭环，再接真实模型。
- Tool Registry 与具体 LLM SDK 解耦。
- 同一 Tool 可被普通 Python Workflow、Tool Calling Loop 和未来 MCP Adapter 复用。
- Trace 能完整重放第一次模型输出、函数执行、结果回填和最终模型输出。
- 与固定 Pipeline 对比；没有路由收益时，不替换当前默认检索链路。

面试问题：

- Tool Calling 为什么不是“LLM 直接执行 Python”？
- Tool Schema、Tool Registry 和 Python Function 分别负责什么？
- 为什么必须保留 `tool_call_id`？
- 如何防止模型调用未注册函数或传入危险参数？
- Function Calling 与 MCP 的关系和区别是什么？

阶段 6：Generation、引用验证与安全

工作内容

- Expected Points 覆盖评价。
- Citation Precision / Recall。
- 验证引用是否支持对应陈述。
- 可回答和不可回答问题的拒答评价。
- 区分证据正确但生成错误的案例。
- 测试上下文截断、重复证据和冲突证据。
- 建立人工抽检报告。

安全回归

- PDF 中的间接 Prompt Injection。
- 用户直接注入。
- 超长 Query。
- 恶意 JSON/Unicode。

- 要求执行外部操作的越权问题。
- 无证据但要求强制回答。

验收门槛

- 每个最终答案可追溯证据。
- 无充分证据时稳定拒答。
- 文档中的指令不能改变系统行为。
- 可区分 Retrieval、Coverage 与 Generation Failure。

面试问题

- 如何评价 Grounded Answer?
 - Citation Correctness 与 Retrieval Recall 有什么区别?
 - RAG 为什么不能自动解决 Prompt Injection?
-

阶段 7: API、LangGraph 与 MCP

7.1 FastAPI

核心 Pipeline 稳定后增加:

- `/query`
- `/health`
- `/versions`
- `/documents/{id}`
- 统一错误码、超时和请求 ID
- CLI、API、测试共用一个核心服务

7.2 LangGraph 的进入条件

只有出现以下需求才迁移:

- 多路状态分支。
- 重试和中断恢复。
- Human-in-the-loop。
- Checkpoint。
- 长任务恢复。

迁移时仅包装现有 Planner、Retriever、Reranker、Generator, 不重写业务组件。必须验证纯 Python Workflow 与 LangGraph 行为一致。

7.3 MCP 的进入条件

核心 API 稳定，并且确实需要被外部 Agent/客户端调用。可暴露：

```
tool: search_bank_documents
resource: bank_document_metadata
prompt: answer_with_bank_evidence
```

MCP Adapter 应复用阶段 5 已验证的 Tool Schema、Tool Registry 和 Python Function。Function Calling 解决的是应用内部“模型请求工具、应用执行并回填结果”的循环；MCP 解决的是工具发现与调用的标准化协议。不要为接入 MCP 再复制一套检索逻辑。

暂时不做

- A2A。
- Multi-Agent。

当前没有多个独立权限域、外部系统和角色协作需求，多 Agent 只会增加调试成本。

验收门槛

- CLI、API、MCP 调用相同核心 Pipeline。
- API/MCP Schema 有契约测试。
- 超时、错误和版本信息可观察。
- LangGraph 必须带来明确的恢复、状态或人工介入价值。

阶段 8：可观测性与推理性能

可观测性

- trace_id。
- Planner、Retriever、Reranker、Merge、Generator Span。
- Query 数、候选数、Reranker Pair 数。
- Prompt、模型、索引、数据集版本。
- P50/P95。
- Failure Type。
- 用户反馈和 Bad Case 回流。

先使用结构化 JSON Trace；只有需要图形化实验管理时再接 OpenTelemetry/Phoenix。

推理性能

- 学习 KV Cache、RoPE、GQA，并做小型 benchmark。

- 为 Generator 建立统一 backend 接口。
- 保留 Hugging Face backend。
- 出现并发服务需求后再增加 vLLM backend。
- 测量 TTFT、TPOT、QPS、GPU 显存和并发。

止损条件

- 单用户场景不为“用了 vLLM”迁移 Linux/WSL。
- 没有负载测试数据时不宣称吞吐提升。

阶段 9: LoRA/QLoRA 与后训练

进入条件

- Prompt 和 Fusion 已充分验证。
- Oracle 有收益，证明 Query Decomposition 有价值。
- LLM Planner 与 Oracle 仍有稳定差距。
- 已积累 Query → Gold Subqueries 数据。
- 有独立 Locked Test。

对照实验

Base + Prompt
Base + Few-shot
LoRA
QLoRA

报告内容

- 复杂问题质量。
- 简单问题退化。
- 格式合法率。
- 延迟。
- 可训练参数。
- 显存。
- Locked Test 泛化效果。

不进入当前主线

- DPO、PPO、GRPO。

除非已经有偏好数据、奖励信号和稳定的生成行为问题，否则把这些内容作为独立学习实验。

5. 学习计划映射

学习内容	是否进入项目主线	合适阶段	原因
Structured Output / Function Calling	是	0、5、7	建立可靠组件协议并手写 Tool Calling 闭环
Agentic RAG / Routing	是	4、5	与 Planner 和补检索直接对应
Agent Evaluation / Reliability	是	1、6	当前项目最缺的能力之一
Context Engineering	是	5、6	处理证据预算、重复、覆盖和引用
ReAct	学习，不直接落地主线	5 前	理解 Agent Loop，但当前应采用受控 workflow
LangGraph	有条件	7	状态、重试和恢复复杂后才有价值
MCP	有条件	7	作为稳定能力的对外适配层
Middleware / Tool Governance	有条件	7	有真实工具后再做权限、超时和审计
KV Cache / RoPE / GQA	学习与实验	8	理解模型推理，不解决检索质量
vLLM / PagedAttention	有条件	8	出现并发吞吐需求后进入
LoRA / QLoRA	有条件	9	必须先证明模型能力是瓶颈
Memory	暂不做长期记忆	未来	单轮协议检索目前不需要用户长期状态
Multi-Agent / A2A	不进入当前主线	未来	缺少真实多角色、多系统需求
DPO/PPO/GRPO	独立学习	未来	当前没有偏好数据和奖励模型问题

6. 全局止损规则

1. 新评测集不能用于证明旧 Baseline 变差；所有版本必须运行相同冻结集合。
2. Oracle 无收益就停止 Planner，不无限调 Prompt。
3. LangGraph 没有带来恢复、状态或可观测性收益就不迁移。
4. MCP 没有真实客户端就不提前接入。
5. vLLM 没有并发负载就不替换当前 backend。
6. LoRA 没有 Locked Test 泛化收益就不保留。
7. 每个新模块必须说明：解决什么失败、相对什么 Baseline、代价是多少。
8. 每阶段只改变一个主要变量，避免无法归因。
9. Tool Calling 没有两个以上有意义的工具或真实路由需求时，不替换固定 Pipeline。
10. Tool Registry 只能执行白名单函数，所有参数和结果必须经过 Schema 校验。

7. 每阶段新对话模板

后续每开启一个阶段的新对话，复制下面的模板：

当前要推进 Project 2 路线的【阶段 X：阶段名称】。

请先：

1. 阅读 docs/project2_long_term_roadmap.pdf 中该阶段的进入条件、验收门槛和止损条件；
2. 检查当前仓库状态和上一阶段产物；
3. 对照真实工程项目或官方文档，说明为什么这样设计；
4. 先给出本阶段最小工作拆分，不要一次完成所有功能；
5. 每次修改后运行与风险相称的测试；
6. 明确哪些结果已经验证，哪些只是推断；
7. 如果实验不如 Baseline，先分析失败层级，不要为了完成阶段强行保留功能。

本阶段目标：

(填写)

本阶段不做：

(填写)

当前已知结果或报错：

(填写)

8. 推荐推进顺序

现在

- 阶段 0: Baseline 与工程基础
- 阶段 1: 版本化评测集
- 阶段 2: 数据和索引版本化
- 阶段 3: Retriever/Reranker 固化
- 阶段 4: Oracle + Planner + Fusion

只有阶段 4 通过

- 阶段 5: Controlled Agentic RAG
 - 手写 Tool Calling Loop / Tool Registry
- 阶段 6: Generation / Citation / Security
- 阶段 7: API / 条件式 LangGraph / MCP
- 阶段 8: Observability / Serving

只有出现模型能力瓶颈

- 阶段 9: LoRA/QLoRA

当前下一步唯一建议：先完成阶段 0 和阶段 1 的设计与最小实现，然后进行阶段 4 所需的 Oracle/Fusion 实验准备。暂时不要实现 LangGraph、MCP、LoRA 或 vLLM。

9. 参考工程与官方资料

- Haystack: <https://github.com/deepset-ai/haystack>
- Haystack DocumentJoiner: https://github.com/deepset-ai/haystack/blob/main/haystack/components/joiners/document_joiner.py
- RAGFlow Quickstart: <https://github.com/infiniflow/ragflow/blob/main/docs/quickstart.mdx>
- RAGFlow Retrieval Testing: https://github.com/infiniflow/ragflow/blob/main/docs/guides/dataset/retrieval_testing.md
- Pyserini: <https://github.com/castorini/pyserini>
- Azure Agentic Retrieval: <https://learn.microsoft.com/en-us/azure/search/agentic-retrieval-overview>
- LangSmith Evaluation: <https://docs.langchain.com/langsmith/evaluation-types>
- LangGraph: <https://www.langchain.com/oss-overview>
- Phoenix: <https://github.com/Arize-ai/phoenix>
- Pydantic Models: <https://github.com/pydantic/pydantic/blob/main/docs/concepts/models.md>
- FastAPI: <https://github.com/fastapi/fastapi>
- OpenAI Function Calling: <https://developers.openai.com/api/docs/guides/function-calling>
- MCP Tools Specification: <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>
- MCP Server Specification: <https://modelcontextprotocol.io/specification/draft/server/index>
- MCP Structured Output: <https://github.com/modelcontextprotocol/python-sdk/blob/main/docs/servers/structured-output.md>
- OWASP Prompt Injection: <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>
- Hugging Face PEFT: <https://huggingface.co/docs/peft/en/methods/overview>
- vLLM GPU Installation: https://docs.vllm.ai/en/latest/getting_started/installation/gpu/